

Discours de présentation (version orale) (environ 11 à 13 minutes)

À lire à voix normale. Les indications entre crochets sont des repères, ne pas les lire.

[Ouverture]

Bonjour à tous. Je vais vous présenter où j'en suis sur mon projet de stage. Je vais d'abord vous expliquer le problème que j'essaie de résoudre, puis comment le système est construit, et enfin ce que j'ai mesuré, avec ses résultats et ses limites. Je vais essayer de rester concret, et de vous dire clairement ce qui est solide et ce qui ne l'est pas encore.

[Le problème]

Le point de départ, c'est une idée simple. Quand un incident arrive sur un réseau, ce qui coûte cher, ce n'est pas seulement la panne. C'est le temps qu'on met à la voir, à comprendre d'où elle vient, et à décider quoi faire.

Il y a trois faits qui guident tout le projet.

Le premier, c'est que les changements de configuration sont une cause majeure de panne. Quand quelqu'un modifie une configuration et se trompe, ça casse. Les études sur le sujet le confirment depuis longtemps. Donc, dans mon système, tout ce qui touche à la configuration et au moment où elle change est traité comme une preuve importante, pas comme un détail.

Le deuxième fait, c'est que dans le temps de réparation, les deux étapes les plus longues, ce sont la détection et le diagnostic. Réparer, une fois qu'on a compris, c'est souvent rapide. Comprendre, c'est ce qui prend du temps. Donc je me concentre exactement sur ces deux étapes.

Le troisième fait, c'est que pour poser un bon diagnostic, il faut croiser des informations très différentes. La configuration, la télémétrie, la topologie, l'état des protocoles de routage. Un opérateur fait ça lentement, en passant d'un outil à l'autre. L'idée, c'est d'automatiser ce croisement.

[Le système, en une phrase]

Alors, ce que je construis, c'est une couche qui se pose au-dessus du jumeau numérique du réseau. Le jumeau, c'est une copie du réseau. Et cette couche fait trois choses. Elle détecte un problème quand il arrive. Elle en cherche la cause. Et elle prévient quand la charge va bientôt dépasser un seuil.

[Les deux voies]

Concrètement, il y a deux voies, et elles ne travaillent pas de la même façon. C'est important de bien les distinguer.

La première voie, c'est du calcul. Je l'appelle la voie quantitative. Elle regarde l'historique de charge, elle y ajoute ce qu'on connaît d'avance, comme les changements de config, les événements, l'heure et le jour, et elle prévoit la charge à venir. Et surtout, elle ne donne pas juste un chiffre, elle donne une fourchette. Quand elle voit qu'un dépassement se prépare, elle produit ce que j'appelle une intention : un dépassement est prévu dans un certain nombre de pas. En résumé, cette voie dit quand et combien.

La deuxième voie, c'est du raisonnement. Je l'appelle la voie agent. Quand un incident est détecté, un raisonneur va chercher la cause. Il appelle des outils, il rassemble des preuves, il propose une cause. Cette voie dit pourquoi.

Et les deux voies se rejoignent. Un dépassement prévu par la première voie, c'est simplement un incident que la deuxième voie traite en avance, avant la panne. C'est le même objet et le même raisonneur des deux côtés. La seule chose qui change, c'est l'origine : soit un problème constaté, soit un problème prévu.

[Le point clé, à dire lentement]

Et j'en arrive au point qui, pour moi, est le cœur du projet. Ce qui distingue mon travail d'un simple appel à une intelligence artificielle, c'est la vérification.

Un modèle de langage peut donner une cause qui a l'air juste. Mais avoir l'air juste, ce n'est pas être juste. Moi, j'ai un jumeau. Donc quand le raisonneur propose une cause, il ne s'arrête pas là. Il rejoue cette cause dans une copie du jumeau. Si les symptômes reviennent, la cause est confirmée. Si les symptômes ne reviennent pas, il cherche ailleurs. Autrement dit, on ne fait pas confiance à une réponse juste parce qu'elle a l'air juste. On la vérifie.

[La voie quantitative, plus en détail]

Je vais maintenant entrer un peu plus dans la voie quantitative, parce que c'est celle que j'ai mesurée ce cycle.

D'abord, d'où viennent les données. Pour l'instant, elles viennent du jumeau simulé. Le jumeau produit, pour chaque lien, une série de télémétrie, avec le taux d'utilisation, les erreurs, la latence. En mode prévision, il tourne avec des pas de cinq minutes, et il utilise un modèle de trafic qui donne à la charge une vraie structure : des pics dans la journée, une différence entre la semaine et le week-end, une croissance lente, et parfois des montées liées à des événements. Tout ça reste reproductible, ce qui est important pour comparer des modèles proprement.

Ensuite, un point sur lequel je veux être très clair, parce que c'est une question de méthode. J'ajoute des variables extérieures, comme les changements de config et les événements. Et quelqu'un pourrait me dire : est-ce que tu ne triches pas, est-ce que tu ne regardes pas la réponse. La réponse est non. À chaque date de prévision, le modèle ne voit que ce qui était connu à ce moment-là. Les événements et les changements futurs sont coupés. Seules les informations de calendrier, comme la date et l'heure, sont autorisées pour le futur, parce que par nature on les connaît d'avance. Pour le dire simplement : le modèle ne voit pas le futur, il utilise seulement ce qu'un opérateur aurait eu sous les yeux au moment de prévoir.

Ce que je compare, ce sont deux modèles simples. Un modèle naïf, qui répète ce qui s'est passé à la même heure le même jour. Et le même modèle, auquel j'ajoute les variables extérieures. L'idée n'est pas de proposer le modèle final. L'idée, c'est d'abord de poser une méthode de mesure sérieuse, et une base de comparaison. Ce deuxième modèle fait le pont entre la couche de configuration et la couche de prévision.

Un mot sur l'architecture, parce qu'elle est en deux étapes. La première étape prévoit la charge future, avec une fourchette. La deuxième étape transforme cette charge future en qualité de service, c'est-à-dire en délai, en perte, en occupation des liens. C'est là qu'un modèle sur graphe, du type RouteNet-Fermi, peut se brancher. Et je veux insister sur un point souvent mal compris : ce modèle sur graphe n'est pas un prédicteur temporel. Ce n'est pas lui qui prévoit le futur. Le futur vient de la première étape. Lui, il traduit un état du réseau en qualité de service. Les deux étapes ont donc des rôles bien séparés.

[Comment je mesure]

Maintenant, comment je mesure tout ça. Parce que le vrai produit de ce cycle, ce n'est pas le modèle, c'est la façon de le mesurer.

Je rejoue la prévision à plusieurs dates de départ, et je regarde la suite. Je ne fais jamais une seule coupe. J'utilise plusieurs horizons, et l'horizon de travail est de vingt-quatre pas, c'est-à-dire deux heures. Pour chaque horizon, j'ai plus de mille sept cents points de prévision.

Pour l'erreur, j'utilise une mesure sans unité, qui se compare au modèle naïf. Quand elle est en dessous de un, ça veut dire qu'on fait mieux que le naïf. Je n'utilise pas le pourcentage d'erreur classique, parce qu'il devient faux quand la charge est basse. Pour la qualité de la fourchette, j'utilise une mesure standard qui récompense à la fois la justesse et la finesse. Et surtout, je ne dis jamais qu'un modèle est meilleur qu'un autre juste en comparant des moyennes. Je fais un test statistique, un test standard qui tient compte du fait que les erreurs successives sont liées entre elles. Comme ça, quand je dis meilleur, ça veut dire vraiment meilleur, pas un hasard.

[Les résultats]

Venons-en aux résultats.

Le résultat principal est clair. Ajouter les variables extérieures améliore la prévision à tous les horizons. À l'horizon de travail, l'erreur baisse fortement. Et le test statistique confirme que la différence est réelle. Quand je retire ces variables, l'erreur remonte tout de suite. Donc ce que ça mesure, c'est directement la valeur de la configuration et des événements. Et c'est aussi, concrètement, le lien entre cette voie et le reste du système. Le gain baisse quand on regarde plus loin dans le futur, ce qui est normal, mais il reste positif partout.

Sur la prévision de dépassement, c'est-à-dire la capacité à dire qu'un seuil va être franchi, le modèle enrichi classe beaucoup mieux les rares dépassements. Mais je veux être honnête là-dessus, et c'est important. Dans ma fenêtre de test, il n'y a que trois dépassements. Trois, c'est peu. Donc le résultat va dans le bon sens, mais il n'est pas encore solide. Il faudra le confirmer sur une période qui en contient beaucoup plus. Je préfère le dire clairement plutôt que de le survendre.

Sur la qualité des fourchettes, à l'horizon de travail, c'est correct. Plus loin dans le futur, les fourchettes deviennent un peu trop étroites. Et ce point est directement lié à quelque chose que j'ai vu sur un cas de dépassement : l'alerte arrive juste au moment du dépassement, pas nettement avant. C'est parce que la fourchette haute, quand elle est trop étroite, met trop de temps à franchir le seuil. Donc améliorer la calibration, c'est aussi ce qui permettra d'alerter plus tôt. Les deux problèmes sont liés.

[Ce qui est solide, ce qui ne l'est pas]

Si je résume, en séparant bien ce qui est solide de ce qui ne l'est pas.

Ce qui est solide repose sur beaucoup de points, plus de mille sept cents par horizon. C'est le classement des deux modèles, et surtout la valeur des variables ajoutées. Ça, j'en suis sûr.

Ce qui n'est pas encore solide repose sur peu de points. C'est la prévision de dépassement, qui n'a que trois événements, et le détail de la calibration horizon par horizon, où il manque de données. Ça ne remet pas en cause le résultat principal, mais ça demande plus de données. Et je préfère être transparent là-dessus.

[La voie agent]

Je reviens maintenant sur la voie agent, la partie diagnostic, parce que je veux expliquer où elle en est.

Aujourd'hui, cette voie est fonctionnelle, mais elle n'est pas encore pilotée par un modèle de langage. La raison est pratique : dans mon environnement de travail actuel, les bibliothèques qui permettent d'appeler ces modèles ne s'installent pas de façon fiable. Donc, pour l'instant, le raisonneur fonctionne avec des règles.

Et je veux insister sur un point. Ce raisonneur à base de règles n'est pas un bricolage, ni une version au rabais. C'est un point de référence. Il rend tout le banc de test reproductible, et il me permet de mesurer ce que l'architecture et le jumeau apportent, avant même d'ajouter un modèle de langage.

D'ailleurs, même avec des règles, la chaîne reste une chaîne d'agent. Parce que ce qui fait l'agent, ce n'est pas le type de raisonneur, c'est la boucle. Le raisonneur ne sort pas une réponse d'un coup. Il choisit quoi vérifier, il appelle un outil, il lit le résultat, il propose une cause, il la rejoue dans le jumeau,

et il conclut avec une cause, des preuves, les clients touchés et un niveau de confiance. Et il fait attention à une chose importante : il distingue la cause de l'effet. Par exemple, une congestion, c'est en général un effet, pas une cause. Donc il ne la prend pas directement comme cause racine.

Le point le plus utile pour la suite, c'est que ce raisonneur à base de règles et le futur raisonneur avec un modèle de langage partagent la même interface. Ils appellent les mêmes outils, et ils produisent le même type de rapport. Donc le raisonneur à règles devient un témoin de comparaison. Quand je brancherai un modèle de langage, je ne me contenterai pas de regarder s'il sait répondre. Je le comparerai dans les mêmes conditions : est-ce qu'il est plus juste, est-ce qu'il appelle moins d'outils, est-ce qu'il conclut plus vite, est-ce que ses preuves tiennent, est-ce que ses causes passent le test du rejeu. Comme ça, j'évite le piège d'un modèle qui a l'air brillant mais dont on ne peut pas vérifier la réponse. Et ça résume bien ma contribution : ce n'est pas d'utiliser un modèle de langage, c'est de construire une boucle de diagnostic qu'on peut vérifier.

[Les limites de périmètre]

Je termine par les limites, pour être complet. Les données viennent encore du jumeau simulé, pas des données réelles du réseau. Et la généralisation que je vise pour l'instant, c'est à l'intérieur d'une même topologie, sur des routes différentes, pas encore à n'importe quel réseau.

[Les prochaines étapes]

Pour la suite, j'ai quatre chantiers. Ajouter des modèles plus complets, dont le modèle sur graphe qui transforme la prévision de charge en prévision de délai et de perte. Traiter la calibration avec une méthode à garantie, ce qui devrait aussi avancer le moment de l'alerte. Brancher les données réelles, ce qui me donnera aussi beaucoup plus de dépassements pour valider cette partie. Et, avec l'équipe Innovation, estimer la matrice de trafic, qui est la donnée qui manque pour le modèle sur graphe.

[Clôture]

Voilà où j'en suis. Pour résumer en une phrase : ce cycle, j'ai mis en place une façon rigoureuse et honnête de mesurer la prévision, j'ai montré que le contexte opérationnel apporte un vrai gain, et j'ai posé une voie de diagnostic vérifiable grâce au jumeau. Je vous remercie, et je réponds à vos questions.